

**National University of Computer and Emerging Sciences
(FAST-NUCES)
PROJECT PROPOSAL**

COMPUTER NETWORKS



**Remote-Access and Site-to-Site VPN with WireGuard — Tunneling,
Overhead and MTU**

Professor: Dr. Ghufraan Ahmed

Submission Date: 04 October 2026

Group Members:

Muzammil Ali – 24K-1023

Syed Muhammad Muzammil – 24K-0887

Abdul Rafay – 24K-0800

1. Problem Statement and Objectives

VPNs secure traffic over untrusted networks, but tunneling adds headers and processing cost that change packet size, effective MTU, latency, throughput and loss behavior. This project studies those networking effects (not only the security function) by building and measuring site-to-site and remote-access VPNs with WireGuard and OpenVPN. Objectives:

- **Build:** a WireGuard site-to-site tunnel between two private networks, a remote-access VPN client, and an equivalent OpenVPN setup (UDP and TCP) for comparison.
- **Analyze:** encapsulation with Wireshark/tcpdump; measure per-packet overhead and the effective tunnel MTU; demonstrate fragmentation, ICMP “fragmentation needed” messages and drops under a wrong MTU.
- **Compare:** baseline, WireGuard, OpenVPN-UDP and OpenVPN-TCP on throughput, latency, loss, CPU use and connection setup, all inside one Linux machine using network namespaces.

2. Proposed Network Architecture

Two private sites (A and B), each with a client behind a VPN gateway, are linked through a router that stands in for the Internet and performs NAT. A remote client also connects over the router to Gateway A. Linux network namespaces emulate every host, gateway and router, so the full topology runs on a single physical computer.

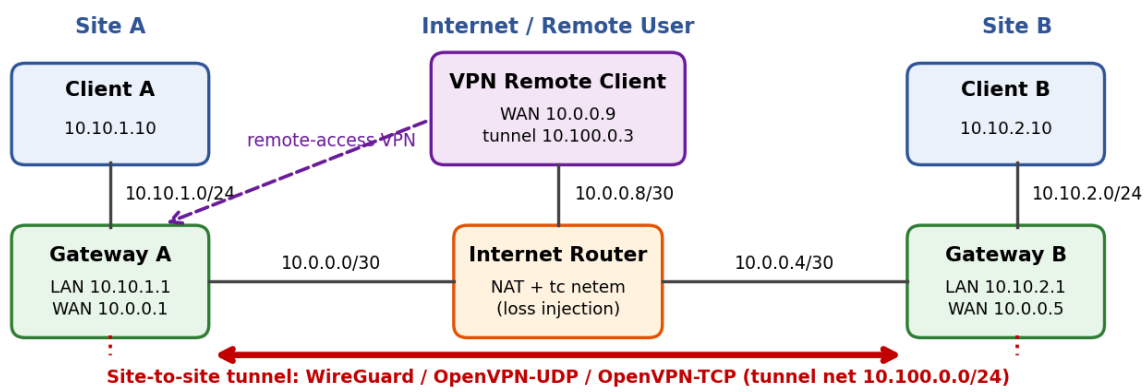


Figure 1. Logical topology and VPN tunnels.

3. Key Protocols and Technologies

Technology	Purpose in this project
IPv4, NAT	Addressing and routing; NAT at the router to mimic Internet gateway behavior.
UDP / TCP	WireGuard and OpenVPN-UDP transport; OpenVPN-TCP comparison; TCP traffic for iperf3.
ICMP, Path MTU Discovery	Connectivity and latency tests; analysis of “fragmentation needed” messages and DF-bit behavior.
WireGuard (Noise-based, UDP)	Primary VPN: site-to-site and remote access.
OpenVPN (TLS control channel)	Comparative VPN, run over UDP and over TCP.
Linux netns, iproute2, tc netem	Virtual hosts/routers and links; netem injects the packet loss.
iperf3, ping, tcpdump/Wireshark	Throughput, latency, packet capture and header analysis.
Bash / Python	Experiment automation, CSV logging, tables and graphs.

4. Addressing Plan

Network	Purpose	Hosts
10.10.1.0/24	Site A LAN	Client A 10.10.1.10; Gateway A 10.10.1.1
10.10.2.0/24	Site B LAN	Client B 10.10.2.10; Gateway B 10.10.2.1
10.0.0.0/30	Gateway A – Router link	Gateway A 10.0.0.1; Router 10.0.0.2
10.0.0.4/30	Gateway B – Router link	Gateway B 10.0.0.5; Router 10.0.0.6
10.0.0.8/30	Remote client – Router link	Remote client 10.0.0.9; Router 10.0.0.10
10.100.0.0/24	VPN tunnel network	Gateway A 10.100.0.1; Gateway B 10.100.0.2; remote client 10.100.0.3

Addresses may be adjusted during implementation; the final plan will be documented in the report.

5. Proposed Proof of Concept (Working Demo)

The demo shows live network behavior, not just configuration files:

- **Demo 1 – Baseline (no VPN):** Client A to Client B through the router; baseline ping and iperf3.
- **Demo 2 – WireGuard site-to-site:** ping between 10.10.1.0/24 and 10.10.2.0/24, routing via the tunnel, handshake status (wg show), iperf3 through the tunnel, and capture showing outer vs. inner headers.
- **Demo 3 – Remote access:** remote client connects, receives 10.100.0.3, reaches Site A/B hosts, and transfers data through the tunnel.
- **Demo 4 – MTU experiment:** deliberately wrong tunnel MTU; show original packet → encapsulation → exceeds link MTU → fragmentation / ICMP / drops in captures, then the corrected MTU.
- **Demo 5 – OpenVPN:** repeat the tests with OpenVPN-UDP and OpenVPN-TCP and compare with WireGuard and the baseline.

6. Experimental Methodology and Evaluation Metrics

Packet loss of 0%, 2% and 5% is injected with tc netem on the router links (base RTT left unchanged). Every configuration is tested under each of the 3 conditions × 3 runs; each run is a fixed-length iperf3 TCP test (same duration and parameters throughout) with a ping series and CPU sampling running in parallel.

Configuration	0% loss	2% loss	5% loss
No VPN (baseline)	3 runs	3 runs	3 runs
WireGuard	3 runs	3 runs	3 runs
OpenVPN-UDP	3 runs	3 runs	3 runs
OpenVPN-TCP	3 runs	3 runs	3 runs

Total: 4 configurations × 3 conditions × 3 runs = 36 runs, reported as mean and spread. Separate packet-size tests use ~64-byte and ~1400-byte payloads plus a DF-bit ping size sweep to find the largest packet that passes.

Metrics: throughput (iperf3, Mbps); latency (ICMP RTT); packet loss and TCP retransmissions; CPU utilization during transfer; VPN overhead = encapsulated size – original size, and overhead % = overhead / original size × 100 (measured from captures, not assumed); MTU behavior (maximum passing packet size, fragmentation, ICMP messages, drops); setup behavior (WireGuard handshake time, OpenVPN connection establishment time).

7. Expected Outcomes

- Captures will show that encapsulation adds measurable per-packet overhead and reduces the effective MTU of the tunnel; a misconfigured MTU will cause fragmentation, ICMP errors or drops.
- Measurable differences in throughput, latency, loss behavior, CPU use and setup time among WireGuard, OpenVPN-UDP, OpenVPN-TCP and the baseline, with OpenVPN-TCP expected to be most sensitive to loss because of TCP-in-TCP retransmissions. These are hypotheses; conclusions will rest on the measured data.
- Deliverables: reproducible namespace + configuration scripts, raw captures and CSV results, graphs/tables, working demo, and a final report.

8. Team Members and Responsibilities

Member (Name, Roll No.)	Role	Main responsibilities
[Muzamil Ali – 24K-1023]	Network infrastructure & WireGuard	Namespace topology, IPv4 addressing, routing and NAT; WireGuard site-to-site and remote-access VPN.
[S.M.Muzammil – 24K-0887]	OpenVPN & MTU experiments	OpenVPN-UDP/TCP setup; MTU, fragmentation and Path MTU Discovery experiments (Demos 4 and 5).
[Abdul Rafay – 24K-0800]	Measurement & analysis	iperf3/ping automation; tc netem loss runs (36 runs); Wireshark/tcpdump analysis; overhead and CPU measurements; graphs and comparison.

All three members share design, debugging, the demo, documentation, the final report and the project defense.

AI-Use Disclosure

Generative AI tools may be used only as supplementary assistance: understanding networking concepts, troubleshooting configuration errors, improving wording of documentation, and reviewing scripts. All configurations, experiments, measurements, analysis and conclusions are implemented, run and verified by the team; AI output is not treated as experimental evidence, and all results come directly from our test environment. This disclosure will be updated to follow the course AI-use policy if it requires.